Derived requirement trace down to procured items

Created 2026-10-07 14:20 CEST

Scope and estimate. The halfarm model traces requirements to blocks by hand (`satisfies`). The user wants the trace carried through the composition to the concrete procured items, and the result imported into a Mission Control view (another agent's repository). User decisions 2026-10-07 (answered before 14:20): (1) the model is the source of the 37 procurement lines and the equipment workbook is generated from it; (2) the derived relation is a separate `<derived>` *affects* edge, `satisfies` stays manual; (3) Mission Control receives a generated `.hymeko` module; (4) first step: the 44 NIST/SIL requirements replace R1–R20 in the model. ETA 4–5 hours. No benchmark.

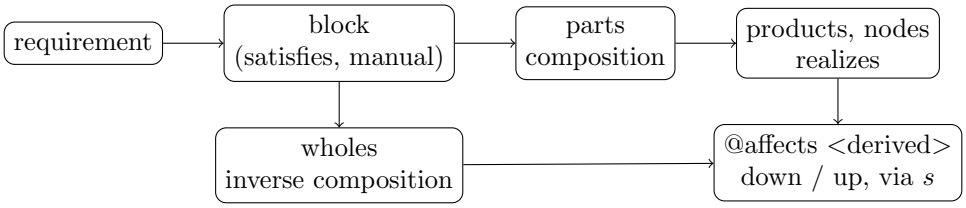
Semantics. For each manual `satisfies` edge $s = (+b, -r)$ two derived n-ary edges are generated: $down = (-r, +x \ldots)$ for every x reachable from b along composition (whole $\rightarrow$ part) and realization (block $\rightarrow$ product or node), and $up = (-r, +y \ldots)$ for every composite y containing b. Field `via` names s. *Affects* means "may be impacted" (change-impact analysis), not "satisfies".

Ownership. Query: `rewrite/scope.rs` gains root items `bind:<sign>:<index|all>` (the current match's binding targets, so a nested `#each` takes its root from the enclosing match) and source items `inverse:<label>` (follow $- \rightarrow +$). `RenderContext` is unchanged (a downstream struct literal in `hymeko_web`). Vocabulary: `data/sysml/meta_sysml_structure.hymeko` gains `priority_level` (with `rank`, as in the Mission Control program vocabulary), `safety_function`, `standard_ref`, `product`, `@cites` (+ requirement, − standard_ref…), `@affects` (+ element…, − requirement). The standard catalog subset and all model content live with the consumer (`02_design/sze/halfarm/model`).

Tests. Unit: `bind:` roots inside a nested `#each`, `bind:+:all`, `inverse:` sources, mixed sources. Counter-tests: `bind:` outside any match resolves no root; `inverse:` of an unknown label follows nothing; without `inverse:` a product is not reached. Consumer: generated requirement table equals the document table (44 rows, every column); generated workbook equals the previous totals and quantities; derived module compiles with the model, regenerating it is byte-identical (freshness check), and removing one composition edge changes exactly the expected derived edges.

Budget and risks. Closure per satisfies edge: $O(\text{iterations} \times \text{edge matches})$, about 60 satisfies edges on a few hundred matches. Risk: *down* over-approximates (every part of a composite); it is labelled *affects*, never *satisfies*. Risk: name resolution of the generated module (cross-file references) – checked by compiling.

State and rollback. Checkout on `feat/material-lobes` with the uncommitted 2026-10-06/07 changes; another worker's files untouched. No git writes from the sandbox. Rollback: revert the `scope.rs` hunks and the vocabulary additions; the consumer files are separate.


requirement
block
(satisfies, manual)
parts
composition
products, nodes
realizes
wholes
inverse composition
@affects <derived>
down / up, via s